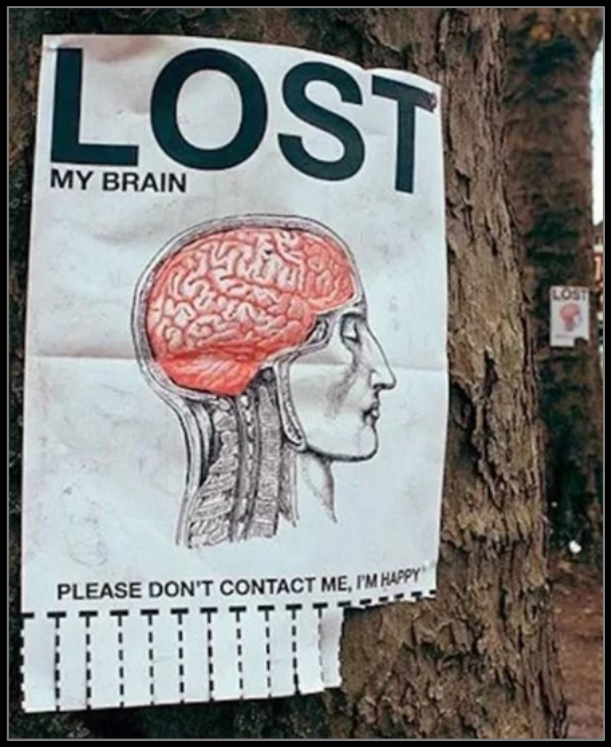
Web Technologies

a data model for the Web

XML family



Prof. Sabin Corneliu Buraga – profs.info.uaic.ro/sabin.buraga/
Assoc. Prof. Andrei Panu – profs.info.uaic.ro/andrei.panu/

“The virtue of the candle lies not in the wax that leaves its trace, but in its light.”

Antoine de Saint-Exupéry

(re)discover...

how do we model data


what does a markup language represent


XML (eXtensible Markup Language)


how do we specify XML namespaces

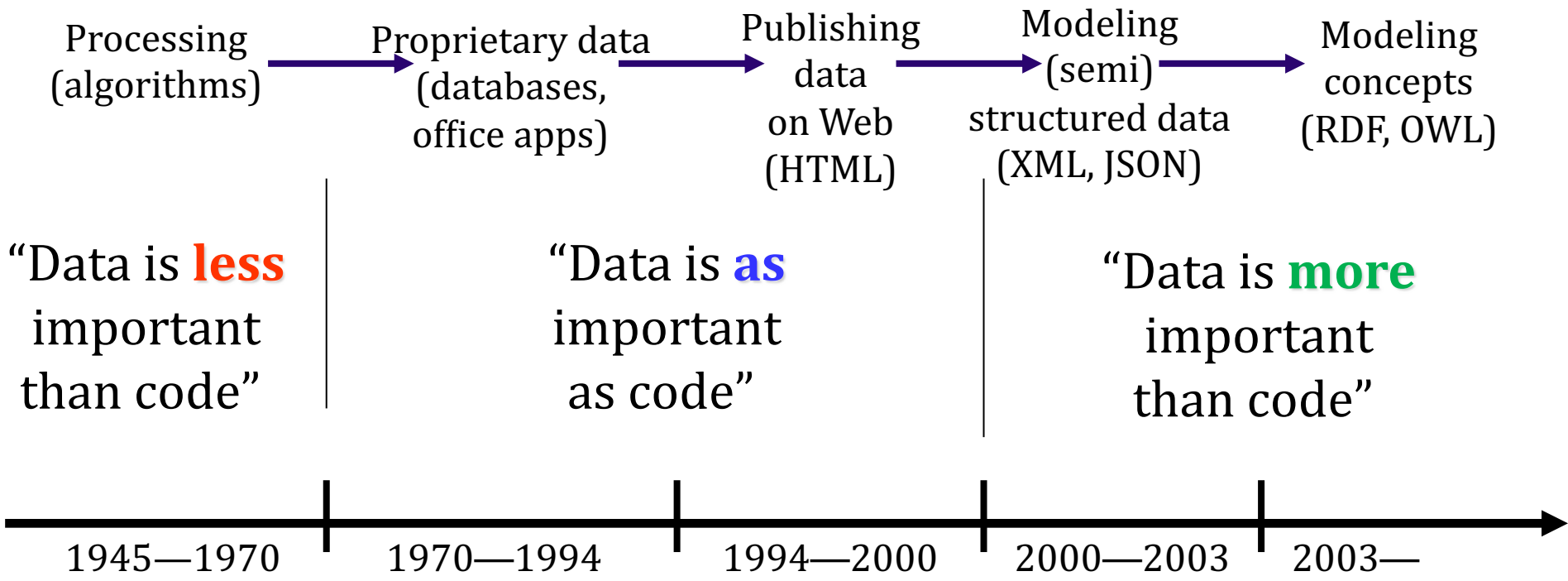
How do we model (represent) data?

preamble

Data must be accessed independently of:

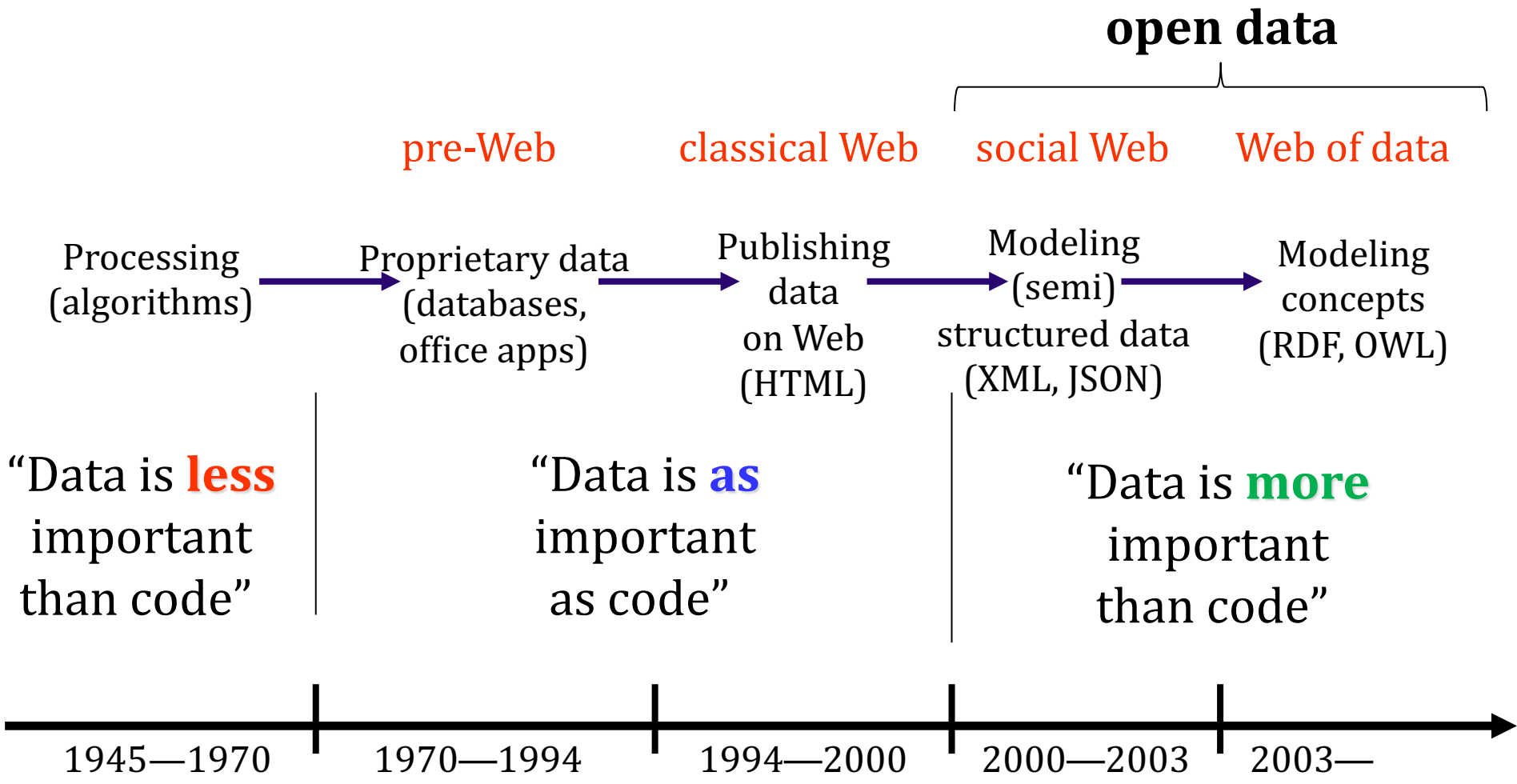
location – file, database management system
manner of representation
serialization format
transmission protocol
operating system
applications which “consume” the data

data: more important than software



evolution of the "data" concept
(adaptation from Daconta *et al.*, 2003)

data: more important than software



evolution of the "data" concept
(adaptation from Daconta *et al.*, 2003)

preamble

Data must be reused and shared on the Web

open data

"a piece of content or data is open
if anyone is free to use, reuse, and redistribute it"
opendefinition.org

Sabin Buraga, *Why 5-Star Data?* (2016)
www.slideshare.net/busaco/why-5star-data

preamble

Data must be reused and shared on the Web

open data

data 🏷️ ≈ wine 🍷

versus

software ⚙️ ≈ fish 🐟

James Governor (2007)

preamble

What data representation model
can we choose for …

storing heterogeneous data from multiple sources?
information that evolves over time?
representing natural language?

preamble

We want to model and process data regarding

poem anthologies
product catalogs of an e-shop
gastronomic recipe repositories
questionnaires
social networks
...

preamble

Necessities:
a language able to annotate information
in an explicit manner

data to be modeled could be practically
unbounded and unknown

no a priori existence of a common vocabulary/schema

preamble

Necessities:
data should be self-explanatory

what does the triple <"Sabin", "Buraga", 30374> mean?

preamble

Necessities:
the adopted model should consider existing navigational architectures, based on hypertext

support for specifying URIs/IRIs

What does a markup language represent?

preamble

Documents:
specific formats *vs.* generic formats

generic encoding (idea from the 1960s):
procedural – calling procedures
based on markups

preamble

GenCode – Stanley Rice, Norman Scharpf (1967)

▼

GML (Generalized Markup Language)

Charles Goldfarb *et al.* (IBM, ~1970)

formal definition of document types

▼

SGML (Standard Generalized Markup Language)

ISO 8879 standard (1986)

preamble: definitions

Markup – annotation, coding

any action denoting explicit interpretation of a text (content) fragment

```php
$finfo = new finfo(FILEINFO_MIME_TYPE);
if (FALSE === $ext = array_search($finfo->file($_FILES['img']['tmp_name']),
    [ 'jpg' => 'image/jpeg',
      'png' => 'image/png',
      'gif' => 'image/gif',
      'webp'=> 'image/webp',
      'svg' => 'image/svg+xml' ], true)) {
    throw new RuntimeException('Upload: format incorect (se                F,
}

// mutam fisierul transferat in directorul cu imagini
// (generam un nume unic pentru fiecare imagine via algoritmul SHA1')
// detalii la https://php.net/manual/en/function.hash.php
$numeImg = sprintf('%s.%s',
                   sha1_file($_FILES['img']['tmp_name']), $ext);
if (FALSE === @move_uploaded_file ($_FILES['img']['tmp_name'],
    IMGDIR . $numeImg)) {
    throw new RuntimeException('Upload: eroare la salvare.');
}

// preluam datele EXIF
$exif = @exif_read_data(IMGDIR . $numeImg, 0, TRUE);
if (FALSE === $exif) {                  // nu exista date EXIF?
    throw new RuntimeException('EXIF: Nu exista date EXIF.');
}
```

special markups

examples:

punctuation marks for written languages (e.g., ¿Vamos?), delimiters used in the source code

preamble: definitions

Specification (annotation, markup) language

markup language

set of markup conventions
used for data encoding

preamble: definitions

Specification (annotation, markup) language

defines the set of mandatory markups,
the manner of how these markups are identified
and structured by using one or more specifications
(i.e. grammars)

xml

Extensible Markup Language

annotation meta-language derived from SGML

W3C standard (1998, 2000, 2004, 2006, 2008)

www.w3.org/TR/xml/

XML TECHNOLOGY

a technology +
a family of languages

On this page → technology topics • news • upcoming events and talks

XML Technologies including XML, XML Namespaces, XML Schema, XSLT, Efficient XML Interchange (EXI), and other related standards.

XML Essentials

XML is shouldered by a set of essential technologies such as the infoset and namespaces. They address issues when using XML in specific applications contexts.

Efficient Interchange

XML standards are omnipresent in enterprise computing, and are part of the foundation of the Web. Because the standards are highly interoperable and affordable, people have wanted to use them in a wide variety of applications. However, in some settings (on devices with low memory or low bandwidth, or where performance is critical) experience has shown that a more efficient form of XML is required.

Schema

Formal descriptions of vocabularies create flexibility in authoring environments and quality control chains. W3C's XML Schema, SML, and data binding technologies provide the tools for quality control of XML data.

Security

Manipulating data with XML requires sometimes integrity, authentication and privacy. XML signature, encryption, and xkms can help create a secure environment for XML.

Transformation

Very frequently one wants to transform XML content into other formats (including other XML formats). XSLT and XPath are very powerful tools for creating different representations of XML content.

Query

XQuery (supported by XPath) is a query language for XML to extract data, similar to the role of SQL for databases, or SPARQL for the Semantic Web.

Components

The XML ecosystem is using additional tools to create a richer environment for using and manipulating XML documents. These components include style sheets, xlink xml:id, xinclude, xpointer, xforms, xml fragments, and events.

Processing

A processing model defines what operations should be performed in what order on an XML document.

Internationalization

W3C has worked with the community on the internationalization of XML, for instance for specifying the language of XML content.

Publishing

XML grew out of the technical publication community.

www.w3.org/XML/

xml: characterization

Descriptive markups

```
<para>
<img />
<response>
<Person>
<tag>
<meta charset="utf-8" />
<MenuItem>
```

xml: characterization

Types of documents

Document Type Definition (DTD)

formal specification of document types (components + structure) – i.e. grammar

used to verify the syntactic correctness

xml: characterization

Data independence

support on every hardware/software platform

XML processors available
for all programming languages

xml: features

Meta-language

able to define other markup languages
markup extension

portable
encoding/language independent via Unicode

xml: features

Solution for content representation of the Web resources identified by URI/IRI

ensuring interoperability (*lingua franca*)

documents are data

xml: constituents

Prologue (preamble)
Elements
Attributes
Entities
CDATA sections
Processing instructions

xml: prologue

Declaration that specifies
document version and encoding

```
<?xml version="1.0" encoding="UTF-8" ?>
```

must appear only once at the beginning of the document

xml: elements

Element = structural component (textual unit)

xml: elements

Element = structural component (textual unit)

name – identifies an element
syntax similar to variable identifiers

product

xml: elements

Syntactically, an element is specified by markups (tags) – start tag and end tag

<product>Ping Uinix</product>

start tag

end tag

xml: elements

Case sensitive

<markup> ≠ <Markup> ≠ <MARKUP>

xml: elements

An element can have an empty content (empty element)

<product></product>

short syntax:

<product />

xml: elements

An element can have an empty content

real examples – HTML specification with XML syntax:

```
<br />
<meta />
<track />
<input />
```

xml: elements

An element can have an empty content

```
<!-- real example: JSX (used by React.js) -->
<Form>
        <Form.Row>
                <Form.Label />
                <Form.Input />
        </Form.Row>
</Form>
```

facebook.github.io/jsx/

xml: content models

Structural model

denotes relations between elements:
sequence, hierarchy, grouping, inclusion

xml: content models

Elements nested in other elements
(could contain textual data and/or other elements)

```
<product>
   Ping Uinix is a
   <obs>multi-colored</obs>
   mascot which sells
   <obs>very quickly</obs>.
</product>
```

<product> (parent node) includes text and <obs> elements (child nodes)

xml: content models

Elements nested in other elements
(could contain textual data and/or other elements)

grouping

```
<!-- HTML5 markups respecting the XML conventions -->
<article>
  <section>
    <ul>
      <li><strong>Stagii pe bune</strong></li>
      <li>Exercism.io</li>
      <li>Code Golf</li>
    </ul>
  </section>
</article>
```

xml: content models

Elements nested in other elements
(could contain textual data and/or other elements)

```
<!-- HTML5 markups respecting the XML conventions -->
<article>
  <section>
    <ul>
      <li><strong>Stagii pe bune</strong></li>
      <li>Exercism.io</li>
      <li>Code Golf</li>
    </ul>
  </section>
</article>
```

hierarchy

sequence

xml: content models

Elements must be correctly closed and nested

** <q>We don't need no education </q>**

```
<?xml version="1.0" ?>
<anthology>
  <poem>
    <title>...</title>
    <stanza>
      <line>...</line>
      <line>...</line>
      ...
    </stanza>
  </poem>
  <poem>
    <title />
  </poem>
  <poem>
    <!-- more poems... -->
  </poem>
</anthology>
```

an XML document
modeling
an anthology of poems

```
<?xml version="1.0" encoding="UTF-8"?>
<products>
   <product>
      <name>Ping Uinix</name>
      <provider>http://www.penguin.info</provider>
      <promo>Mascot of the month</promo>
   </product>
   <product>
      <!-- a sort of blue oranges -->
      <name>Blue Ory</name>
      <description />
   </product>
   <product>
      <name>⌚ having taste of 🍎</name>
   </product>
</products>
```

semi-structured data

▼

flexibility

a possible product catalog used by an e-shop

xml: attributes

Attribute

describes a specific property (characteristic) about a particular occurrence of an element

xml: attributes

Attributes are only specified inside a start tag

```
<anthology status= "draft" date="2026-04-09">
  ...
</anthology>
```

```
<student xml:id="0314159265" account="Tu.Pi">
  <name initial= "I">Tuxy Pinguinnescool</name>
</student>
```

xml: attributes

Attributes can be defined in any order

```
<Button id="@+id/sync_settings" text="@android:string/ok" />
```

≡

```
<Button  text="@android:string/ok" id="@+id/sync_settings" />
```

real examples: Android applications

android.googlesource.com/platform/packages/apps/

xml: attributes

Attribute names are case sensitive

```
<img src="..." />
```

≠

```
<img SRC="..." />
```

xml: attributes

Attribute value is mandatory to be delimited by the quotes "…" or the apostrophes '...'

attributes with no value are not accepted

xml: attributes

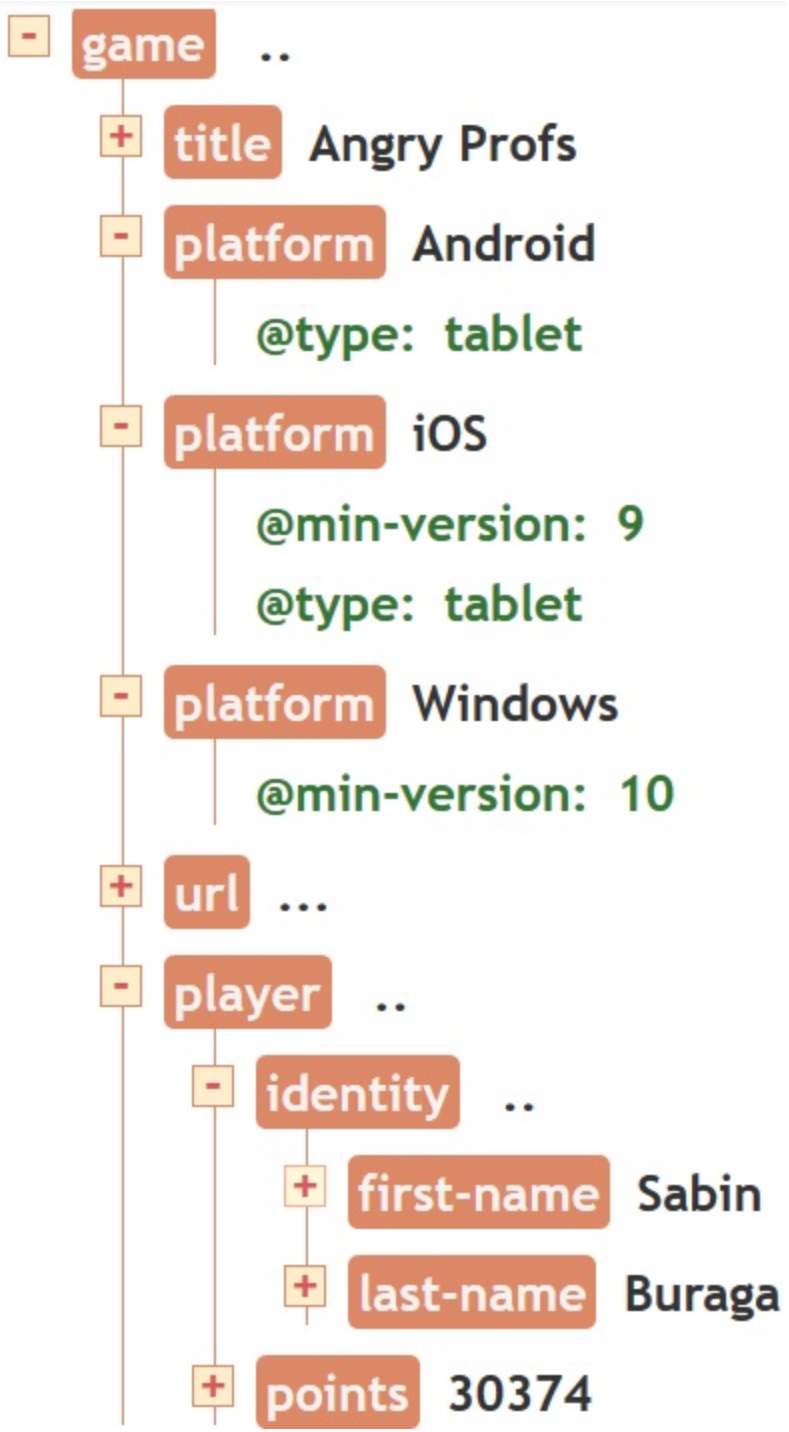
```
<form action=process.php method="GET'>
<label for=search">Search:</label>
<input default type=search placeholder= /></form>
```


```
:</label> Expected attribute's quoted string value.
type=search placeholder= /></form>
```

```
<game>
  <title>Angry Profs</title>
  <platform type="tablet">Android</platform>
  <platform min-version="9" type="tablet">iOS</platform>
  <platform min-version="10">Windows</platform>
  <url>…</url>
  <player>
    <identity>
       <first-name>Sabin</first-name>
       <last-name>Buraga</last-name>
       <!-- possibly, other info… -->
    </identity>
    <points>30374</points>
    …
  </player>
</game>
```

XML data regarding an electronic game

visualizing the hierarchical structure of XML data



codebeautify.org/online-xml-editor

xml: entity references

Goal:
coding and referring a part of a document

syntax:
&identifier;
or
&#number;

xml: entity references

Default entities – similar to HTML ones:

< (<) **>** (>) **&** (&) **"** (")

Character entity references:

** ** (non-breaking space – ** ** for HTML)

ă (“ă” – ISO-8859-2 and Unicode character sets)

❀ (“❀” symbol – Unicode)

xml: sections

Certain parts of a document
could require a special treatment

CDATA – excludes the XML processing

xml: sections

```
<script type="application/javascript">

if (visits < 10) { // not a regular visitor
   $("#message").html ("<p>Hi!</p>");
}

</script>
```

```
if (visits < 10) { // not a regular visitor
------------^
```

XML Parsing Error: not well-formed
Line Number 3, Column 13

xml: sections

```
<script type="application/javascript">
/*<![CDATA[*/
if (visits < 10) { // not a regular visitor
   $("#message").html ("<p>Hi!</p>");
}
/*]]>*/
</script>
```

the XML processor will not interpret the syntax of JS code

xml: processing instructions

Include information regarding (external) applications to be invoked for content processing

<?*processing-instruction* ... ?>

Example:
associating a CSS stylesheet
for rendering the content of an XML document

```
<?xml-stylesheet type="text/css" href="styles.css" ?>
```

```
/* formatting XML data abount an electronic game */
* { display: block; font-family: sans-serif; }
game { margin: 2em; }
title { font-size: 2em; font-weight: bold; color: navy; }
platform { display: inline; font-size: 0.9em; color: gray; }
identity::before { content: "User: "; }
first-name, last-name { display: inline; font-style: italic; }
url, points { display: none; }
```

Angry Profs

Android iOS Windows
User: *Sabin Buraga*

xml: space processing

White spaces – e.g., space, TAB, NL (New Line) or CR (Carriage Return) characters – have no significance

```
<VisualAsset id="obsObject">
  <enabled>true</enabled>
  <zOrder>0</zOrder>
  <Orientation>
    <roll>90</roll>
    <tilt>90</tilt>
    <heading>90</heading>
  </Orientation>
</VisualAsset>
```

≡

```
<VisualAsset id="obsObject">
<enabled>true</enabled><zOrder>0
</zOrder><Orientation><roll>90</roll>
<tilt>90</tilt><heading>90</heading>
</Orientation></VisualAsset>
```

ARML (Augmented Reality Markup Language) markups

xml: overview

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<!-- pentru redarea datelor, se recurge la o foaie de stiluri CSS -->
<?xml-stylesheet type="text/css" href="game.css" ?>
<game>
    <title>Angry Profs</title>
    <platform type="tablet">Android</platform>
    <platform min-version="9" type="tablet">iOS</platform>
    <platform min-version="10">Windows</platform>
    <url>...</url>
    <player>
        <identity>
            <first-name>Sabin</first-name>
            <last-name>Buraga</last-name>
            <!-- ... -->
        </identity>
        <points>30374</points>
    </player>
</game>
```

an XML document is composed of node types: elements, attributes, comments, processing instructions, document type definition (DTD)

xml: overview

preamble

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<!-- pentru redarea datelor, se recurge la o foaie de stiluri CSS -->
<?xml-stylesheet type="text/css" href="game.css" ?>

    <platform type="tablet">Android</platform>
    <platform min-version="9" type="tablet">iOS</platform>
    <platform min-version="10"
    <url>...</url>
    <player>
        <identity>
            <first-name>Sabin</first-name>
            <last-name>Buraga</last-name>
            <!-- ... -->
        </identity>
        <points>30374</points>
    </player>
</game>
```

comment node

processing instruction

attribute node

start tag

end tag

text node (characters)

an XML document is composed of node types: elements, attributes, comments, processing instructions, document type definition (DTD)

(instead of) break

www.instagram.com/pawel_kuczynski1

xml: family

XML (Extensible Markup Language)

syntax

XML Information Set – Infoset

(abstract) data model

XLL (Extensible Linking Language)

XLink – links between documents

XPointer – resource relative locators

XSL (Extensible Stylesheet Language)

transforming & formatting: XSLT + XSL-FO

XQuery (with XPath)

XML data querying

xml: usages

Content structuring/formatting
(data presentation formats)

 in the Web browser:
(X)HTML (Extensible HTML), HTML5
www.w3.org/TR/html/

 vector graphics:
SVG (Scalable Vector Graphics)
www.w3.org/Graphics/SVG

xml: usages

Representation of various types of content

 mathematical expressions: MathML

www.w3.org/Math/

 synchronized multimedia data:

SMIL (Synchronized Multimedia Integration Language)

www.w3.org/TR/SMIL/

xml: usages

Representation of various types of content

 voice dialogue information: VoiceXML

voicexml.org

 cartographic info: KML (Keyhole Markup Language)

developers.google.com/kml/

 hydrologic data: WaterML

www.ogc.org

xml: usages

Representation of various types of content

🪄 user-interface components:

Android User Interface Layouts

developer.android.com/guide/topics/ui/declaring-layout

FXML (JavaFX)

github.com/mhrimaz/AwesomeJavaFX

QML (*Qt Meta-object Language*)

doc.qt.io/qt-6/qmlapplications.html

XAML (Extensible Application Markup Language)

docs.microsoft.com/en-us/windows/uwp/xaml-platform/

xml: usages

Representation of various types of content

 documentations:

DocBook (Documentation Book)

docbook.org

 info processed by office applications – e.g., Open Office:

ODF (Open Document Format)

www.oasis-open.org/committees/tc_home.php?wg_abbrev=office

xml: usages

Representation of various types of content

📡 Web syndication – newsfeeds:
RSS (Really Simple Syndication)
www.rssboard.org/rss-specification
Atom Syndication Format
datatracker.ietf.org/doc/html/rfc4287

📖 electronic publications (e-books): EPUB
github.com/w3c/epub-specs/

xml: usages

Representation of various types of content

🩺 medical information (EHR – Electronic Health Records)
HL7: **www.hl7.org/implement/standards/**

💰 electronic businesses
FpML–Financial products Markup Language: **www.fpml.org**

👤 governmental information
NIEM–National Information Exchange Model: **niem.github.io**

xml: usages – other domains

BeerXML
BDML (Biological Dynamics Markup Language)
CAP (Common Alerting Protocol)
CML (Chemical Markup Language)
COLLADA (COLLAborative Design Activity)
DFXML (Digital Forensics XML)
GPX (GPS Exchange Format)
MEI (Music Encoding Initiative)
RTML (Remote Telescope Markup Language)
SSML (Speech Synthesis Markup Language)
STAR (Standards for Technology in Automotive Retail)
TEI (Text Encoding Initiative)

What if we choose names of markups/attributes already defined by other XML-based languages?

```
<event uri="https://stagiipebune.ro/">
    <name xml:lang="ro">Stagii pe Bune</name>
    <year>2026</year>
</event>
```

```
<participant>
    <name uri="mailto:tux@info.uaic.ro">
        Tuxy Pinguinnesscool</name>
    <year kind="Bachelor">2</year>
</participant>
```

```
<event uri="https://stagiipebune.ro/">
   <name xml:lang="ro">Stagii pe Bune</name>
   <year>2026</year>
</event>
```

conflict!

?

```
<participant>
   <name uri="mailto:tux@info.uaic.ro">
      Tuxy Pinguinnesscool</name>
   <year kind="Bachelor">2</year>
</participant>
```

<name> event name ≠ **<name>** person name

<year> calendaristic year ≠ **<year>** study year

xml: namespaces

XML namespace

denotes a vocabulary used to qualify – in a unique way – the XML elements/attributes

xml: namespaces

The defined vocabulary – a collection of element and attribute names, plus their structure – could be denoted by an URI

xml: namespaces

The defined vocabulary could be denoted by a URI

xmlns attribute specifies that URI;
optionally, assigns a unique identifier
to each used vocabulary

W3C specification:
www.w3.org/TR/xml-names/

```
<c:calendars xmlns:c="http://www.calendar.info">
 <e:participant xmlns:s="http://www.info.uaic.ro/Students/"
                xmlns:e="http://www.info.uaic.ro/Events/">
  <s:name>Tuxy Pinguinnescool</s:name>
  <s:year s:kind="Bachelor">2</s:year>
  <c:calendar>
   <e:event xml:id="SpB">
    <e:name xml:lang="ro">Stagii pe Bune</e:name>
    <e:year>2026</e:year>
   </e:event>
   <e:event xml:id="GSoC">
    <e:name xml:lang="en">Summer of Code</e:name>
  </c:calendar>
 </e:participant>
</c:calendars>
```

no conflicts!

study the examples from
the archive associated to this lecture

xml: namespaces – examples

The HTML vocabulary: **http://www.w3.org/1999/xhtml**

The SVG namespace: **http://www.w3.org/2000/svg**

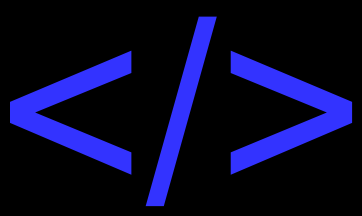
The Vocabulary of the XML *Sitemaps* documents :
http://www.sitemaps.org/schemas/sitemap/0.9

The Vocabulary of the XML Schema data types:
http://www.w3.org/2001/XMLSchema

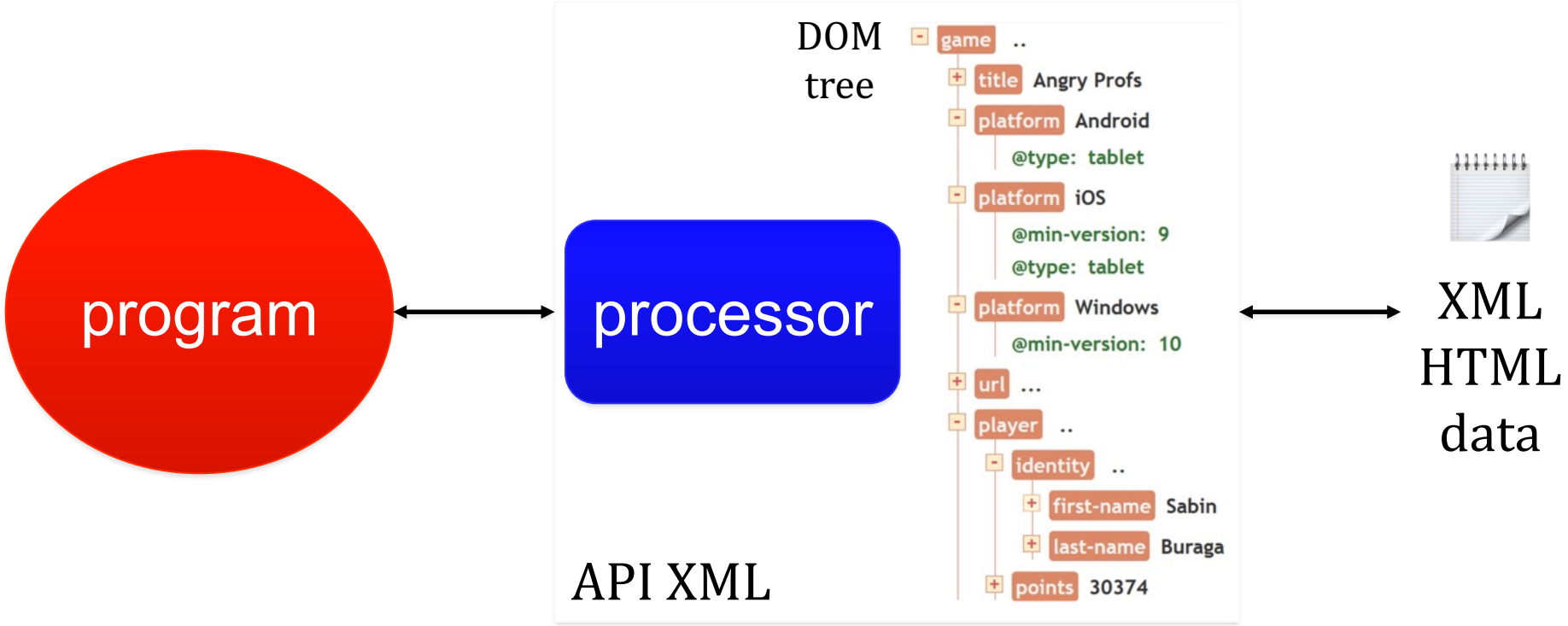
The namespace for Android applications:
http://schemas.android.com/apk/res/android

summary

XML data modeling



characterization, uses, XML namespaces,



next episode:

XML/HTML document processing using DOM